Lecture 28.08: Common Pruning Rules & Hyperparameter Regularization

📌 Executive Summary

In-depth study of Scikit-Learn tree regularization hyperparameters: max_depth, min_samples_split, min_samples_leaf, max_leaf_nodes, and min_impurity_decrease.

🏛️ Theoretical Formulation & Mathematical Architecture

🛠️ Scikit-Learn Pre-Pruning Hyperparameters

To enforce regularized inductive bias, Scikit-Learn provides five core structural hyperparameter constraints:

Hyperparameter	Mathematical Definition	Default	Typical Tuned Range	Engineering Impact
`max_depth`	$\max\{\text{depth}(m) \mid m \in \mathcal{L}\} \leq d_{\max}$	`None` (unbounded)	$[3, 10]$	Hard ceiling on hierarchy depth; drastically cuts variance
`min_samples_split`	$\lvert\mathcal{D}_t\rvert \geq s_{\min}$ to attempt split	`2`	$[10, 100]$	Prevents splitting small, statistically unreliable sample subsets
`min_samples_leaf`	$\min(\lvert\mathcal{D}_L\rvert, \lvert\mathcal{D}_R\rvert) \geq l_{\min}$	`1`	$[5, 50]$	**Highest impact**: guarantees statistical support at

Hyperparameter	Mathematical Definition	Default	Typical Tuned Range	Engineering Impact
				all terminal leaves
`max_leaf_nodes`	$\|\mathcal{L}\| \leq L_{\max}$	`None`	$[10, 64]$	Best-first tree growth; bounds overall capacity
`min_impurity_decrease`	$\Delta I(t) \geq \tau_{\min}$	`0.0`	$[10^{-4}, 10^{-2}]$	Rejects splits that provide negligible information gain

---

Why `min_samples_leaf` is Superior to `min_samples_split`

Consider a node with 20 samples (`min_samples_split=20`):

- A split producing $|\mathcal{D}_L| = 19$ and $|\mathcal{D}_R| = 1$ is valid under `min_samples_split=20` .
- However, $|\mathcal{D}_R| = 1$ is a dangerous single-sample leaf that memorizes noise.
- Setting `min_samples_leaf=10` forces both children to contain at least 10 samples, instantly disqualifying asymmetric noise splits.

Vectorized Python Implementation

```
from sklearn.tree import DecisionTreeClassifier

# Regularized Tree using Pre-Pruning Hyperparameters
regularized_tree = DecisionTreeClassifier(
    max_depth=5,
    min_samples_split=20,
    min_samples_leaf=10,
    random_state=42
)

# Fits with strict structural safeguards
regularized_tree.fit(X_syn, y_syn)
print("Regularized Tree Depth:     ", regularized_tree.get_depth())
```

```
print("Regularized Leaf Count:     ", regularized_tree.get_n_leaves())
assert regularized_tree.get_depth() <= 5, "Depth constraint must be strictly respecte
```

🔬 Key Engineering Takeaways & Invariants

1. **Axis-Aligned Partitioning**: Decision trees recursively split feature space along orthogonal boundaries without requiring feature scaling.
2. **Impurity Optimization**: Split selection deterministically maximizes impurity reduction (Information Gain for classification, Variance Reduction for regression).
3. **Generalization Safeguards**: Regularization via pre-pruning (`max_depth` , `min_samples_leaf`) and post-pruning (`ccp_alpha`) prevents noisy leaf memorization.